

ФГБОУ ВО «Пензенский государственный университет»

Факультет вычислительной техники

Кафедра «Вычислительная техника»

ОТЧЕТ

по дисциплине «Логика и ОА»

лабораторная работа №2 на тему:

«Динамические списки»

Выполнил студенты группы 25ВВВ3:

Коновалов А. С

Шорохов В. В

Приняли:

к.т.н., доцент Юрова О. В.

к.т.н., доцент Деев М. В.

Тема: «Динамические списки»

Цель работы: изучить динамические структуры данных на языке C, освоить работу с односвязными списками, указателями и динамическим выделением памяти (malloc, free). Реализовать на основе односвязного списка три структуры данных: приоритетную очередь, обычную очередь (FIFO) и стек (LIFO). Освоить операции добавления, удаления, поиска и просмотра элементов.

Лабораторное задание:

Задание 1: Реализовать приоритетную очередь путём добавления элемента в список в соответствии с приоритетом объекта (объект с большим приоритетом становится перед объектом с меньшим приоритетом).

Задание 2: На основе приведённого кода реализовать структуру данных «Очередь».

Задание 3: На основе приведённого кода реализовать структуру данных «Стек».

Пояснительный текст:

Задание 1. Приоритетная очередь

структуру node добавлено поле int prior — приоритет объекта. Список поддерживается упорядоченным по убыванию приоритета.

Ключевая функция spstore() отличается от базовой: элемент не всегда добавляется в конец. Логика вставки:

если список пуст или приоритет нового элемента больше приоритета головы — элемент становится головой;

иначе проходим по списку, пока приоритет следующего элемента больше или равен приоритету нового, и вставляем после найденного узла.

При равных приоритетах новый элемент встаёт после уже существующих. Остальные функции (get_struct, review, find, del) совпадают с базовым примером, только в get_struct добавлен ввод приоритета.

Сложность: вставка — $O(N)$, поиск и удаление по имени — $O(N)$.

Результат работы программы:

```
--- Приоритетная очередь ---  
Имя - Fisher, приоритет - 2  
Имя - Fish, приоритет - 1  
-----
```

Задание 2:

Очередь работает по принципу «первым пришёл — первым ушёл».

Добавление — только в конец, извлечение — только из начала.

`enqueue()` — добавляет элемент в конец. Если очередь пуста, элемент становится одновременно головой и хвостом; иначе присоединяется после `last`, и `last` переносится на него.

`dequeue()` — извлекает элемент из начала: сохраняет указатель на голову, выводит значение, переносит `head` на следующий элемент, освобождает память. Если очередь опустела, `last` обнуляется.

Сложность: добавление и извлечение — $O(1)$ благодаря указателям `head` и `last`.

Результат работы программы:

```
--- Очередь (FIFO), начало слева ---  
rider  
home  
-----
```

Задание 3:

Программа реализует стек на базе односвязного динамического списка.

Стек работает по принципу LIFO (Last In, First Out): последним пришёл — первым ушёл. И добавление, и извлечение происходят с одного конца — вершины стека.

Структура элемента совпадает с базовым примером из методички: поле `inf[256]` для полезной информации и поле `nextr` для ссылки на следующий

элемент. В отличие от очереди, здесь достаточно одного указателя `top` — на вершину стека. Указатель `last` не нужен, так как работа с хвостом не ведётся.

Функция `get_struct()`. Создаёт новый элемент списка. Через `malloc` выделяется память под структуру; при неудаче выводится сообщение об ошибке и программа завершается через `exit(1)`. Запрашивается название объекта; если введена пустая строка, память освобождается и возвращается `NULL`. Поле `next` обнуляется.

Функция `push()`. Кладёт элемент на вершину стека:

новый элемент создаётся через `get_struct()`;

его поле `next` указывает на бывшую вершину `top`;

указатель `top` переносится на новый элемент.

Вставка выполняется за $O(1)$ без прохода по списку.

Функция `pop()`. Снимает элемент с вершины:

если стек пуст — выводится сообщение «Стек пуст»;

иначе сохраняется указатель на текущую вершину, её значение выводится на экран, `top` переносится на следующий элемент, память освобождается через `free`.

Функция `peek()`. Позволяет посмотреть значение вершины без извлечения. Полезно, когда нужно узнать верхний элемент, но не удалять его.

Функция `review()`. Выводит содержимое стека от вершины к основанию.

Функция `main()`. Реализует меню: `push`, `pop`, `peek`, просмотр стека, выход. В начале `main` вызывается `setlocale(LC_ALL, "Russian")` — это обеспечивает корректное отображение русских символов в консоли Visual Studio.

Результат работы программы:

```
--- Стек (LIFO), вершина сверху ---
e
f
-----
```

Вывод:

В ходе работы были изучены динамические структуры данных на языке C и освоена работа с односвязными списками через указатели и функции malloc/free. На основе единого базового кода реализованы три структуры: Приоритетная очередь — список, упорядоченный по убыванию приоритета. Отличие от базового примера — модифицированная функция spstore(), которая ищет место вставки, а не добавляет в конец.

Очередь (FIFO) — добавление в конец (enqueue), извлечение из начала (dequeue). Обе операции за $O(1)$ благодаря указателям head и last.

Стек (LIFO) — добавление и извлечение через вершину top, все операции за $O(1)$. Экспериментально подтверждено, что динамические списки позволяют эффективно распределять память: элементы создаются по мере необходимости и освобождаются сразу после удаления.